Ch 10: Working with Instances and OOP Design

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Working With Instances

Object Instances

Each instance of a class has its own set of data attributes.

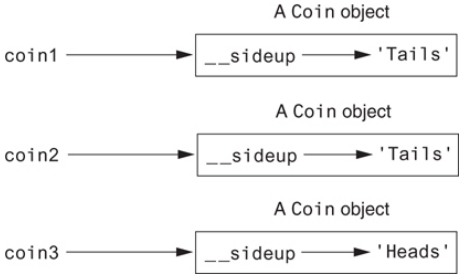


Figure 1: Three instances of Coin class after toss can have different sides up (Gaddis, 2021, pg.566)

- **Instance attribute**: belongs to a specific instance of a class
  - Created when a method uses the `self` parameter to create an attribute
- If many instances of a class are created, each would have its own set of attributes

```
# three Coin instances, all will be
# initialized to Heads
coin1 = Coin()
coin2 = Coin()
coin3 = Coin()


# after tossing, each can be on a
# different side, because each instance
# has separate __sideup attribute
coin1.toss()
coin2.toss()
coin3.toss()
```

EAST TEXAS A&M

Accessor and Mutator Methods

- Typically, all of a class's data attributes are private and provide methods to access and change them.
- **Accessor methods**: return a value from a class's attribute without changing it
  - Safe way for code outside the class to retrieve the value of attributes
- **Mutator methods**: store or change the value of a data attribute
  - Again safe because since attribute is hidden, the method can check that new value is valid and legal before setting

Note: Mutator methods are sometimes called "setters" and accessor methods are sometimes called "getters"



Passing Objects as Arguments

- Methods and functions often need to accept objects as arguments
- When you pass an object as an argument, you are actually passing a reference to the object
  - The receiving method or function has access to the actual object
  - Methods of the object can be called within the receiving function or method
  - Data attributes may be changed using mutator methods

```
def show_coin_status(coin_obj):
    """
    Example of a Coin object passed as
    a parameter to a regular function.
    """
    print('The side of the coin that is up: ',
      coin_obj.get_sideup())

my_coin = coin.Coin()
show_coin_status(my_coin)
```

EAST TEXAS A&M

Techniques for Designing Classes

- **UML Diagram**: (Unified Modeling Language) standard diagrams for graphically depicting object-oriented systems
- General layout: box divided into three sections
  - Top: name of class
  - Middle: list of class attributes
  - Bottom: list of class methods

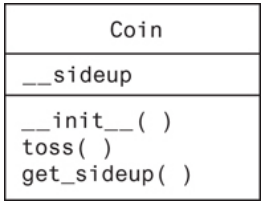


Figure 2: UML diagram for the Coin class (Gaddis, 2021, pg.587)

EAST TEXAS A&M

Finding the Classes in a Problem

When developing object-oriented program, first goal is to identify classes

Typically involves identifying the real-world objects that are in the problem

1. Get written description of the problem domain
   - Should include physical objects that need to be simulated, roles of people, recordkeeping items
2. Identify all nouns in the description, each of which is a potential class
   - Should include noun phrases and pronouns
3. Refine the list to include only classes that are relevant to the problem
   - Remove duplicates and synonyms
   - Remove items that are not relevant to the program
   - Remove nouns that represent objects, not classes

EAST TEXAS A&M

Identifying a Class's Responsibilities

- A classes responsibilities are:
  - The things the class is responsible for knowing
    - Identifying these helps identify the class's data attributes
  - The actions the class is responsible for doing
    - Identifying these helps identify the class's methods
- To find out a class's responsibilities look at the problem domain
  - Deduce required information and actions

EAST TEXAS A&M

Summary





References

Gaddis, T. (2021). *Starting out with python* (5th). Pearson.

EAST TEXAS A&M